

créer un environnement de travail + le site de base :

```
C:\Users\karab>cd "C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation"
```

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation>mkdir tp2-docker
```

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation>cd tp2-docker
```

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>mkdir site
```

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>cd site
```

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker\site>notepad index.html
```

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>TP Docker</title>
</head>
<body>

  <h1>Mon serveur Docker fonctionne</h1>
  <p>TP 2 conteneurisation</p>

</body>
</html>
```

on creer le dockerfile

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker\site>cd ..
```

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>notepad Dockerfile
```

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

```
FROM nginx:alpine
RUN echo "Configuration Docker OK"
WORKDIR /usr/share/nginx/html
COPY site/index.html .
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Le Dockerfile sert à construire une image Docker personnalisée.

Le code du Dockerfile permet d'utiliser une image nginx légère grâce à nginx:alpine, puis de copier notre fichier index.html dans le serveur web nginx afin que notre page soit automatiquement affichée dans le conteneur Docker.

on construit l'image docker:

- docker build -t mon-site .

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker build -t mon-site .
[+] Building 7.4s (8/8) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 111B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine 2.1s
=> [auth] library/nginx:pull token for registry-1.docker.io    0.0s
=> [internal] load .dockerignore                                0.1s
=> => transferring context: 2B                                   0.0s
=> [internal] load build context                                0.2s
=> => transferring context: 288B                                 0.0s
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:2f07d83bf561b506400dc183b1b2003803e39efbd 3.3s
=> => resolve docker.io/library/nginx:alpine@sha256:2f07d83bf561b506400dc183b1b2003803e39efbd 0.1s
=> => sha256:94de7c34b8d25a6e2241d9f9af135eba064193cecalb6b782e80d9d8738c24 20.28MB / 20.28MB 1.2s
=> => sha256:0faf356544319b35bb9aa03595e556e22277228565e7470fa7da4b1a2c999aa5 1.39kB / 1.39kB 0.2s
=> => sha256:86e0029eccff957b151ddd0fb727628e9ae5fcde85cb9771e74d471efe031245 1.21kB / 1.21kB 0.3s
=> => sha256:bcace0045394f92e9a583eb4f11a674f91cc91d407b85d52afe3f6209672ca1e 400B / 400B 0.3s
=> => sha256:04a7ff7b108a8f7d67888aa7f32283bcda09520660a1d149bd4e6a294cf8b135 951B / 951B 0.1s
=> => sha256:2c084cd9252c2e4b305bd605f0b6534943993f25adfb9c92dec27d9f2dcca45a2 625B / 625B 0.1s
=> => sha256:16e661a44239bffc66a00e3a0d6c45af7059317a60a5685760937a68d0b3b624 1.88MB / 1.88MB 0.3s
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB 0.5s
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 0.3s
=> => extracting sha256:16e661a44239bffc66a00e3a0d6c45af7059317a60a5685760937a68d0b3b624 0.3s
=> => extracting sha256:2c084cd9252c2e4b305bd605f0b6534943993f25adfb9c92dec27d9f2dcca45a2 0.1s
=> => extracting sha256:04a7ff7b108a8f7d67888aa7f32283bcda09520660a1d149bd4e6a294cf8b135 0.1s
=> => extracting sha256:bcace0045394f92e9a583eb4f11a674f91cc91d407b85d52afe3f6209672ca1e 0.1s
=> => extracting sha256:86e0029eccff957b151ddd0fb727628e9ae5fcde85cb9771e74d471efe031245 0.1s
=> => extracting sha256:0faf356544319b35bb9aa03595e556e22277228565e7470fa7da4b1a2c999aa5 0.1s
=> => extracting sha256:94de7c34b8d25a6e2241d9f9af135eba064193cecalb6b782e80d9d8738c24e8 0.6s
=> [2/2] COPY site/index.html /usr/share/nginx/html/index.html 0.4s
=> exporting to image                                           0.9s
=> => exporting layers                                           0.5s
=> => exporting manifest sha256:3282edd49341504297a26641c1a39f3e7d632030d3f15734a9e3b913843d3 0.1s
=> => exporting config sha256:ecf8fe9bbca0ad162fc0622c7e3ce63948ab3c290994b44e03cb48af7ad17cd 0.1s
=> => exporting attestation manifest sha256:36d10ca45e37917637c0c937b2701620278200db1c94dfa48 0.1s
=> => exporting manifest list sha256:76312e6676bfa102a14a11df2bf58d9daf224bbb30c0a987da95acdb 0.1s
=> => naming to docker.io/library/mon-site:latest              0.0s
=> => unpacking to docker.io/library/mon-site:latest           0.1s

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>|
```

creer un reseau docker personnalise:

- docker network create mon-reseau

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker network create mon-reseau
e131af39163a7433c6b9423a7f8b5b64921aacb5a8683c89320cba473f4a41d

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>|
```

on verifie:

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker network ls
NETWORK ID      NAME      DRIVER  SCOPE
8a928b48d682    bridge    bridge   local
0260c974b56d    host      host     local
e131af39163a    mon-reseau bridge    local
51990579827a    none      null     local

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

créer un volume docker:

- docker volume create mon-volume

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker volume create mon-volume
mon-volume

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

vérifier:

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker volume ls
DRIVER  VOLUME NAME
local   mon-volume

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

lancer le conteneur:

- docker run -d --name mon-nginx --network mon-reseau -v mon-volume:/usr/share/nginx/html -p 8080:80 mon-site

-d → arrière-plan

--name → nom du conteneur

--network → utilise ton réseau personnalisé

-v → volume persistant

-p 8080:80 → localhost:8080 vers nginx port 80

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker run -d --name mon-nginx --network mon-reseau -v mon-volume:/usr/share/nginx/html -p 8080:80 mon-site
65d040d12b528c49bb2985335452a3e389e8446f8a607838dc13306f7d515032

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

on vérifie:

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker ps
CONTAINER ID  IMAGE      COMMAND                  CREATED        STATUS        PORTS
65d040d12b52  mon-site   "/docker-entrypoint...."  50 seconds ago Up 49 seconds  0.0.0.0:8080->80/tcp
cp, [::]:8080->80/tcp
f083e39ae778  ubuntu    "bash"                  2 hours ago   Up 2 hours
ubuntu-test

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

on tape l'url : http://localhost:8080/



on vérifie le conteneur:

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
0fe045923688   mon-site   "/docker-entrypoint...." 43 seconds ago Up 42 seconds 0.0.0.0:8080->80/t
cp, [::]:8080->80/tcp   mon-nginx
f083e39ae778   ubuntu    "bash"                  2 hours ago   Up 2 hours
ubuntu-test

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

on vérifie les logs:

```

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker logs mon-nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/20 09:48:05 [notice] 1#1: using the "epoll" event method
2026/05/20 09:48:05 [notice] 1#1: nginx/1.31.0
2026/05/20 09:48:05 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/05/20 09:48:05 [notice] 1#1: OS: Linux 6.6.114.1-microsoft-standard-WSL2
2026/05/20 09:48:05 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/05/20 09:48:05 [notice] 1#1: start worker processes
2026/05/20 09:48:05 [notice] 1#1: start worker process 30
2026/05/20 09:48:05 [notice] 1#1: start worker process 31
2026/05/20 09:48:05 [notice] 1#1: start worker process 32
2026/05/20 09:48:05 [notice] 1#1: start worker process 33
2026/05/20 09:48:05 [notice] 1#1: start worker process 34
2026/05/20 09:48:05 [notice] 1#1: start worker process 35
2026/05/20 09:48:05 [notice] 1#1: start worker process 36
2026/05/20 09:48:05 [notice] 1#1: start worker process 37
2026/05/20 09:48:05 [notice] 1#1: start worker process 38
2026/05/20 09:48:05 [notice] 1#1: start worker process 39
2026/05/20 09:48:05 [notice] 1#1: start worker process 40
2026/05/20 09:48:05 [notice] 1#1: start worker process 41
172.18.0.1 - - [20/May/2026:09:48:14 +0000] "GET / HTTP/1.1" 200 216 "-" "Mozilla/5.0 (Windows NT 10.0
; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "-"

What's next:
  View and search logs for all containers in one place
  with Docker Desktop's Logs view. docker-desktop://dashboard/logs

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>

```

on vérifie le réseau personnalisé :

```

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
8a928b48d682        bridge              bridge              local
0260c974b56d        host                host                local
e131af39163a        mon-reseau          bridge              local
51990579827a        none                null                local

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>

```

vérifier le volume:

```

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker volume ls
DRIVER      VOLUME NAME
local       mon-volume

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>

```

on vérifie l'image docker:

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
mon-site:latest	76312e6676bf	92.7MB	26MB	U
nginx:latest	800e7c98538c	241MB	66MB	
rockylinux/rockylinux:latest	f3c70d748f4c	293MB	75.7MB	U
ubuntu:latest	f3d28607ddd7	160MB	45.3MB	U

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

vérifier les détails du conteneurs:

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker inspect mon-nginx
```

```
[
  {
    "Id": "0fe045923688b7b1881f6bc6e82b23e8b4cb6c7a4033d92929fc40c69802e335",
    "Created": "2026-05-20T09:48:05.363730633Z",
    "Path": "/docker-entrypoint.sh",
    "Args": [
      "nginx",
      "-g",
      "daemon off;"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 1172,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-05-20T09:48:05.566501073Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:76312e6676bf102a14a11df2bf58d9daf224bb30c8a987da95acd8c487dbd5",
    "ResolveConfPath": "/var/lib/docker/containers/0fe045923688b7b1881f6bc6e82b23e8b4cb6c7a4033d92929fc40c69802e335/resolve.conf",
    "HostnamePath": "/var/lib/docker/containers/0fe045923688b7b1881f6bc6e82b23e8b4cb6c7a4033d92929fc40c69802e335/hostname",
    "HostsPath": "/var/lib/docker/containers/0fe045923688b7b1881f6bc6e82b23e8b4cb6c7a4033d92929fc40c69802e335/hosts",
    "LogPath": "/var/lib/docker/containers/0fe045923688b7b1881f6bc6e82b23e8b4cb6c7a4033d92929fc40c69802e335/0fe045923688b7b1881f6bc6e82b23e8b4cb6c7a4033d92929fc40c69802e335-json.log"
  },
  {
    "Name": "/mon-nginx",
    "RestartCount": 0,
    "Driver": "overlayfs",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "",
    "ExecIDs": null,
    "HostConfig": {
      "Binds": null,
      "ContainerIDFile": "",
      "LogConfig": {
        "Type": "json-file",
        "Config": {}
      },
      "NetworkMode": "mon-reseau",
      "PortBindings": {
        "80/tcp": [
          {
            "HostIp": "",
            "HostPort": "8080"
          }
        ]
      },
      "RestartPolicy": {
        "Name": "no",
        "MaximumRetryCount": 0
      },
      "AutoRemove": false,
      "VolumeDriver": "",
      "VolumesFrom": null,
      "ConsoleSize": [
        51,
        102
      ],
      "CapAdd": null,
      "CapDrop": null,
      "CgroupsMode": "private",
      "Dns": null,
      "DnsOptions": null,
      "DnsSearch": null,
      "ExtraHosts": null,
      "IpcMode": "private",
      "Isolation": "default",
      "Links": null,
      "MaxOpenFiles": 0,
      "MaxPid": 0,
      "Network": "mon-reseau",
      "Pidfile": "",
      "Privileged": false,
      "ReadOnlyRootfs": false,
      "SecurityOpt": null,
      "StorageOpt": null,
      "Sysctl": null,
      "Tmpfs": null,
      "Ulimits": null,
      "Userns": "default",
      "WorkingDir": ""
    },
    "Name": "/mon-nginx",
    "RestartCount": 0,
    "Driver": "overlayfs",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "",
    "ExecIDs": null,
    "HostConfig": {
      "Binds": null,
      "ContainerIDFile": "",
      "LogConfig": {
        "Type": "json-file",
        "Config": {}
      },
      "NetworkMode": "mon-reseau",
      "PortBindings": {
        "80/tcp": [
          {
            "HostIp": "",
            "HostPort": "8080"
          }
        ]
      },
      "RestartPolicy": {
        "Name": "no",
        "MaximumRetryCount": 0
      },
      "AutoRemove": false,
      "VolumeDriver": "",
      "VolumesFrom": null,
      "ConsoleSize": [
        51,
        102
      ],
      "CapAdd": null,
      "CapDrop": null,
      "CgroupsMode": "private",
      "Dns": null,
      "DnsOptions": null,
      "DnsSearch": null,
      "ExtraHosts": null,
      "IpcMode": "private",
      "Isolation": "default",
      "Links": null,
      "MaxOpenFiles": 0,
      "MaxPid": 0,
      "Network": "mon-reseau",
      "Pidfile": "",
      "Privileged": false,
      "ReadOnlyRootfs": false,
      "SecurityOpt": null,
      "StorageOpt": null,
      "Sysctl": null,
      "Tmpfs": null,
      "Ulimits": null,
      "Userns": "default",
      "WorkingDir": ""
    }
  }
]
```

Communication entre 2 conteneurs:

créer le buntu:

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker run -dit --name ubuntu-test --network mon-reseau ubuntu bash
```

```
5e128eff696432058b2b728198bb17f0b2f10b181a1ccd0641ace6f9a8f6b3bb
```

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>
```

on vérifie et on exécute:

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
5e128eff6964   ubuntu        "bash"                  45 seconds ago Up 45 seconds
0fe045923688   mon-site      "/docker-entrypoint..." 13 minutes ago Up 13 minutes  0.0.0.0:8080->80/tcp
cp, [::]:8080->80/tcp  mon-nginx

C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker exec -it ubuntu-test bash
root@5e128eff6964:/# |
```

on update :

```
C:\Users\karab\OneDrive\Documents\cours bachelor\conteneurisation\tp2-docker>docker exec -it ubuntu-test bash
root@5e128eff6964:/# apt update
Get:1 http://archive.ubuntu.com/ubuntu resolute InRelease [136 kB]
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [136 kB]
Get:3 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [50.1 kB]
Get:4 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [136 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [48.9 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [27.9 kB]
Get:7 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [136 kB]
```

on installe le ping:

```
root@5e128eff6964:/# apt install iputils-ping -y
Installing:
  iputils-ping

Installing dependencies:
  libcapi2 libidn2-0 libunistring5 linux-sysctl-defaults

Summary:
  Upgrading: 0, Installing: 5, Removing: 0, Not Upgrading: 2
  Download size: 761 kB
  Space needed: 2677 kB / 1024 GB available
```

La commande `apt install iputils-ping -y` permet d'installer l'outil réseau ping dans le conteneur Ubuntu afin de tester la communication entre plusieurs conteneurs connectés au même réseau Docker personnalisé.

on ping:

```
root@5e128eff6964:/# ping mon-nginx
PING mon-nginx (172.18.0.2) 56(84) bytes of data.
64 bytes from mon-nginx.mon-reseau (172.18.0.2): icmp_seq=1 ttl=64 time=0.133 ms
64 bytes from mon-nginx.mon-reseau (172.18.0.2): icmp_seq=2 ttl=64 time=0.241 ms
64 bytes from mon-nginx.mon-reseau (172.18.0.2): icmp_seq=3 ttl=64 time=0.194 ms
^C
--- mon-nginx ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2016ms
rtt min/avg/max/mdev = 0.133/0.189/0.241/0.044 ms
root@5e128eff6964:/#
```

Les deux conteneurs ont été connectés au même réseau Docker personnalisé grâce à l'option `--network mon-reseau`. Docker leur attribue automatiquement une adresse IP interne et un DNS interne, ce qui permet au conteneur `ubuntu-test` de communiquer avec `mon-nginx` via la commande `ping mon-nginx`.

schéma:

[Navigateur]

|

localhost:8080

|

[mon-nginx]

|

mon-reseau

|

[ubuntu-test]

Ce TP nous a permis de comprendre la création d'images Docker, le fonctionnement des réseaux Docker, la communication entre conteneurs ainsi que la persistance des données avec les volumes.